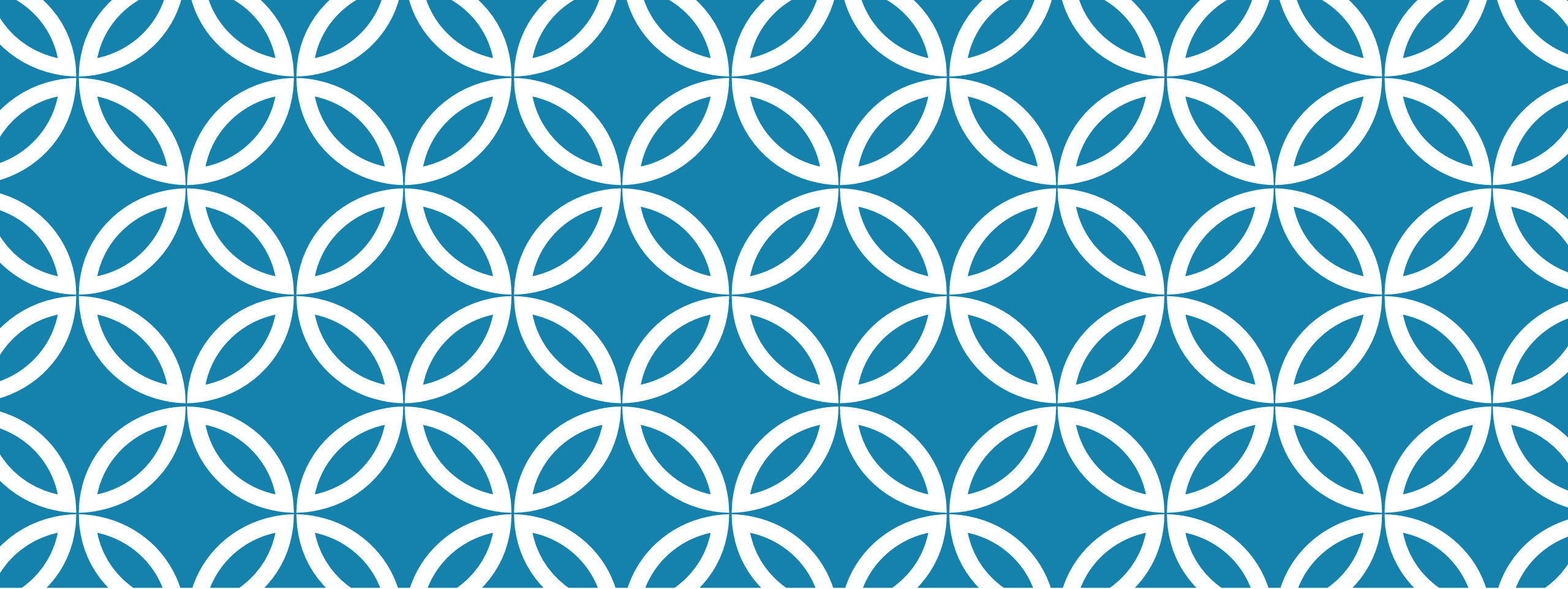




WHATSAPP SECURITY

Application layer security.

By: Dr. Ramzi Saifan

INTRODUCTION

قبل ما يُرسل من الجهاز
يكون encrypted ولا واحد
بالتالي بقدر يقرأ ما ويغيرها.

- WhatsApp uses **end-to-end encryption** ^{sender Receiver} ↑
- Users may have **multiple devices** Registered to the same account.
- **Each device** has its **own encryption keys** public & private keys [pairs of keys]
- ^{Private} **Key** is not known by **the server**, and the server **can't cheat** the public key I send to server only the PK Key.
- **Any manipulation** of the **public key** is **easily detectable** بال PK
تقدرش يلعبة
تبعك. بقدرش
يعطيك واحد False
لا توبعطيك معاه
evidence ستأخذ صار
ال PK راجع لـ هـ الشخص
مفي ملحوظة فيه

PRIMARY DEVICE AND KEY TYPES

➤ Primary device - A device that is used to register a WhatsApp account with a phone number.

- This primary device can be used to link additional companion devices to the account. *add others device by scan QR code.*

لأول مرة
الدخول
الرجستير

➤ Public Identity Key, which is a long-term, Curve25519

- Is an Elliptic Curve private/public keys pair
- The key pair is generated at install time.

→ client generate pairs of key using elliptic curve algorithm.

* elliptic curve :- Asymmetric encryption.
PU } same as RSA
PR } PU is known to all
PR just me.

NEW USER REGISTRATION

➤ When a user installs WhatsApp and registers a phone number:

➤ **Key Generation on the Device**

- The client generates a **long-term identity key pair** using the **Signal Protocol** (Curve25519).
- The **private key stays on the device**. *In private + secure place.*
- The **public identity key** is sent to WhatsApp's server, where the server receives the tuple: (**User Identifier** → **Public Identity Key**) *no phone number*
- The user identifier is typically the **phone number or account ID**. *to server I send phone number + public key & hide my private key.*

PUBLIC KEY STORAGE

يوجد ال PU عند ال
- Server

➤ In WhatsApp Messenger, the **user's public key** is **stored in two places logically**:

1. The **operational key directory (server database)**
2. The **transparency log (Merkle tree)**

➤ Merkle tree **does not store the key**. Instead, it stores a **cryptographic commitment to the key** so clients can verify integrity.

hash(PU + user-id) ➤ The **primary operational key directory** used for messaging. [Encryption messaging]

*The key is saved in 2DB:-

*Key directory DB:- called sparse merkle tree

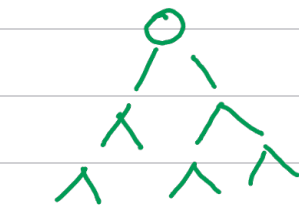
phone number
079

compute hash (user ID)

PU وين انجزت ال index: 079 XXX

data stored:- public keys

PU وين انجزت ال index - وين انجزت ال وين انجزت ال وين انجزت ال



if multiple devices store multiple keys.

* Transparency log DB:- called merkle tree

stored action -> log (sequential): every new key is stored in the log (append-only)

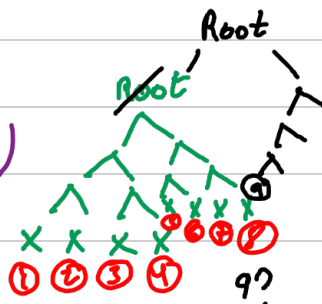
ما بعينه القديم .

index: Sequential

view 19

data stored:- hash (user ID, public key)
phone #

stored sequentially ~



directory $\rightarrow$ stored PU keys.

Transparency $\rightarrow$ hash (userID, public ID) / log.

← بالاول اذا عملت على PU من بعد overwork لاننى تفى ال Index
 ← اذا كنت على multiple devices يكون مفتوح multiple keys.

- * Key directory used to retrieve public keys of users $\rightarrow$ end to end.

* Transparency log for audit & integrity $\rightarrow$ انا كد انوال key
میں محبوب $\rightarrow$
من قبل ال server / الحكومة / بالذ

↳ guarantees nobody play public key

KEY TRANSPARENCY

➤ **Key Transparency** is a mechanism that allows users to verify that the public keys used in end-to-end encryption have not been secretly replaced by the service provider or an attacker.

➤ It protects against:

- Server-side key substitution
- Malicious insider attacks
- Government key replacement
- Infrastructure compromise

غیر-خلد

➤ When Alice wants to message Bob, she fetches Bob's public key from the server database.

عشان اعرم ل key تاملے بجیک من ال key directory

Transparency → hash, Not the keys

KEY SUBSTITUTION ATTACK

Directory DB

② عشان اچھو ماسک اول خطوہ بدی ال your PU بجیب م ال
 (اگرچہ لکھو) اچھو ال PU تاہ۔ ال server اذا اعطانی تاہ لہا تاہی

➤ If the server is compromised, it could give Alice a fake key.
 server بے موافقہ ال Data لا

➤ Alice encrypts to attacker's key instead of Bob's.

➤ The attacker can decrypt messages.

➤ This is called a **key substitution attack**.

key transparency log DD prevents this kind of attack (KSA):-

➤ **Solution:** every user's public key is recorded in a public, append-only, verifiable cryptographic transparency log (Merkle Tree).

➤ If a key is changed or replaced secretly:

- It becomes detectable. ^{why?} ① tree is audited
 - It cannot happen silently. ② different versions
 - ③ append-only when go to next stage
- key ال نہ بدلے ال key tree سے ال نہ ہو ال

key directory → key

transparency → evidence that PU
 مں ملوہ ہے۔ ال ال ال اور بات۔

① whatsapp on the device generates pair of keys. we send to the server phone number + public key.
 How we guarantee that whatsapp doesn't send the private key also?

→ whatsapp gives its design to **third party** to make sure everything works well.

→ we are guarantee application will not send private key.

WHERE MERKLE TREE FITS

DB: evidence proof
 this key belongs to this person

➤ **Merkle tree log** stores a **verifiable record** of keys or key mappings.

- Merkle Tree Log

Leaf1: Hash(User1 || PublicKey1)

Leaf2: Hash(User2 || PublicKey2)

Leaf3: Hash(User3 || PublicKey3)

➤ The log is usually implemented as a **Merkle tree where each leaf corresponds to a log entry.**

➤ Each entry represents an **event**, such as: **key registration**, **key update**, and **device change**.
 (add device ①, ②, ③ all corresponds to keys)

➤ The tree provides: **Integrity**, **Auditability**, **Tamper detection**

➤ Suppose Bob installs WhatsApp Messenger twice.

- First device: Entry1 : Bob → KB1

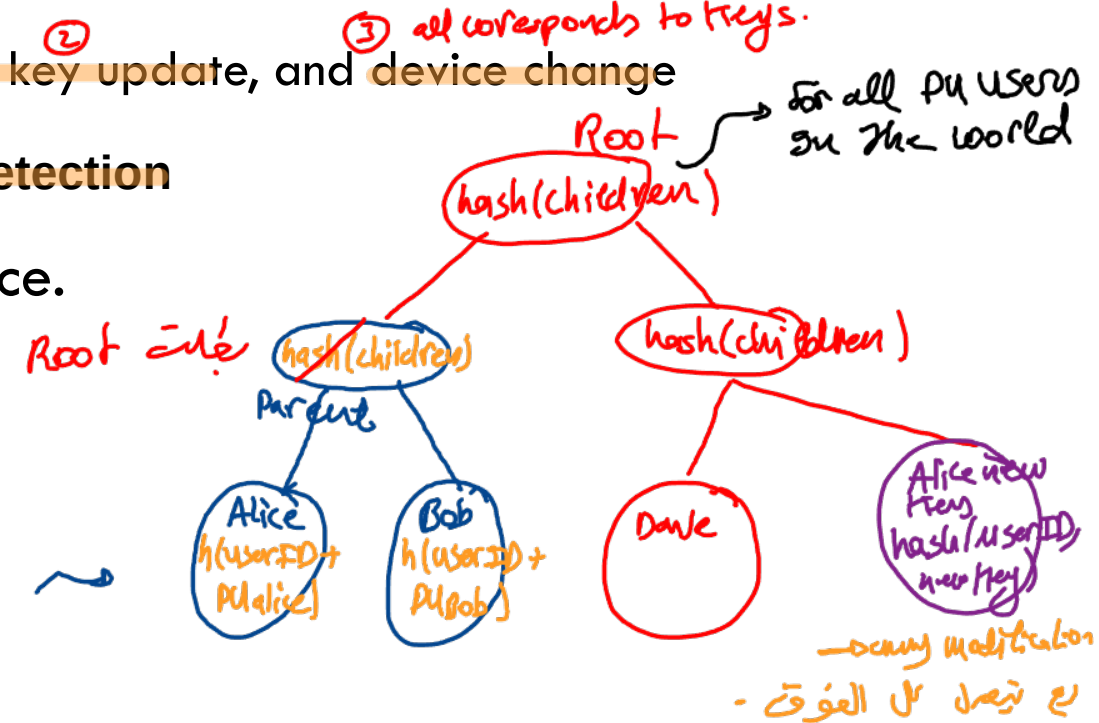
- Second device: Entry3 : Bob → KB2

each one has its own log.

➤ Bob has two leaves in the **append-only log**.

all info are in the leaves

~ who verify this DB? & verify its consistent - append only-?
 → each installation for whatsapp or moved from device to device generates new pair of keys. **directory** → just save new key & if theres multiple devices all keys saved in it
 auditing → transparency is key all amazon cloud is open source
 → you append only new key, without deleting the old one



DATA STRUCTURES

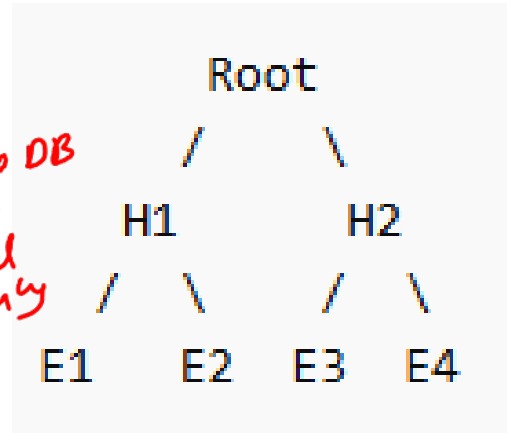
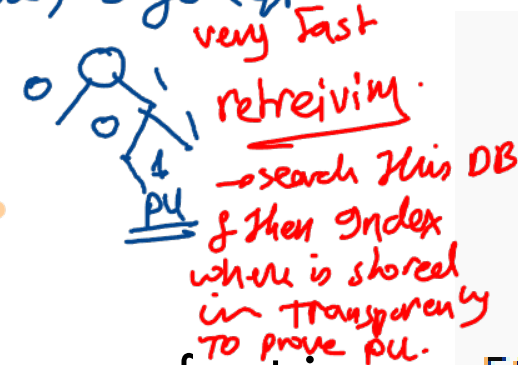
1. **Key Directory Tree:** for key lookup, the system builds a **sparse Merkle tree** indexed by the hash of the user ID.

- index = Hash(UserID) 256 bits

- E.g., index = Hash(079xxxxxxxx)

- Each phone number has one entry, which contains the public key(s)

1 → go Right, 0 go left
Index 1101



2. **Append Transparency log:** It is built simply as a sequence of entries:

- Entries appear in **chronological order**, not by user hash.

- Entry_i = Hash(UserID || PublicKey) who to search?

stored sequentially.



Directory stores PU + Transparency tree index: 2000
⇒ complexity $O(\log)$, as you modify all the up

• two DBs بال updates التالى و بكونوا بال
• two DBs published queue و تعديل ال epoch بعد التغيير ال two DBs

ADDING NEW KEY

1. In the Transparency log tree

- Leaf Creation:** A leaf node is created containing: $\text{hash}(\text{userID} || \text{public_key})$
- Leaf Insertion:** The leaf is appended to the Merkle tree log sequentially.
- The index of the leaf node is sent and stored in the Key directory sparse Merkle tree
- Tree Recalculation:** Hashes propagate upward: $\text{parent_hash} = \text{hash}(\text{left_child} || \text{right_child})$
- New Merkle Root:** The tree produces a new root hash representing the entire log state.
- Signed Tree Head:** The server signs the root hash:
 - $\text{SignedTreeHead} = \text{Sign}(\text{root_hash}, \text{timestamp})$
 - This prevents undetected log rewriting

ترتيب تسلسلي
ال (level) القوت.

Digital signature

$E(\text{hash}(\text{Root}))$

PR_{server}

Root hash و Root Digital sig بعثت مع ال
كا بعثت ال User-ID بعثت مع ال

2. In the sparse Merkle Key directory Tree

- Instead of appending entries sequentially, the key is modified and the position is: $\text{leaf_index} = \text{hash}(\text{user_identifier})$.
- Example: $\text{hash}(+962XXXXXXXXXX) \rightarrow$ binary path in the tree
- Store the public key and the index of the log event

يعني و مثال

وين هتوز بال transparency

HOW TO VERIFY THE LOG

Suppose the system has 4 users: Alice, Bob, Carol, and Dave.

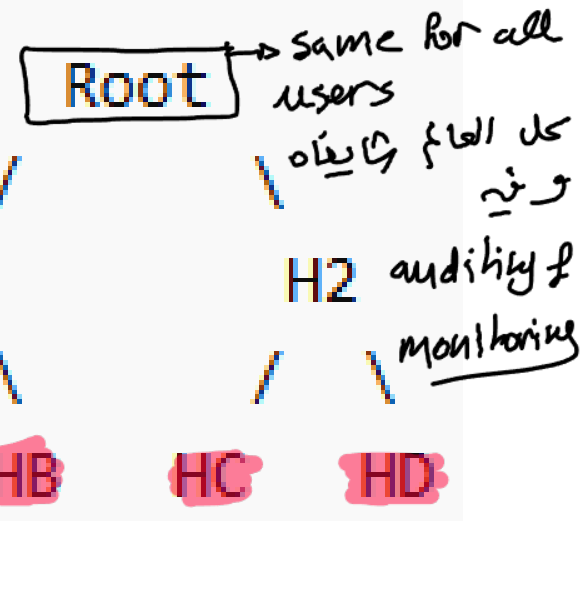
- HA = Hash(Alice || KA)
- HB = Hash(Bob || KB), and so on

Alice بتبعته ل server رقم تلفون Bob ال server به Key ال Key
 بآ Bob بلا فقه بال Key directory . بعديه ال PU ل Bob منو بعته
 مع ال PU ال Value ال سiblings . بعته ال PU ل Bob وبعته
 HA و H2 و hash(Root) وال Root digital signature .

For Alice to verify Bob's public key, the server sends to Alice as proof:
 the neighbor nodes needed to recompute the root.

- Sibling hash 1 = HA
- Sibling hash 2 = H2
- Root hash = R

① verify hash
 مش مطلوب فيه
 decryption PU server
 وال hash سويود فينقلنا . لرح متأكد لو ال Root hash
 صدا فقه فيه .



② HB بعته ال [رقم تلفون + PU Bob] بحب HB
 وبعته معاه HA فبحب H1 و صديق
 احب hash[Root] .

⇒ bit (log entry [users])
 ↓
 30 levels

*when sending the server phone number it sends ① hash(Root) Digital signature
 ➤ If $R' = R$: then the proof is valid.
 ② PU to one I want to talk
 ③ all hash for siblings
 ↳ # = log [entry]

MALICIOUS SERVER

ممكن ان لا PU حتى مطلوب في بحث ال hash على توفيق لا
Root اننا مستخدم ال hash لل Root و نضع نفس ال hash
ال كجمل في ال . معناه — هذا ال Key في مطلوب في

➤ The malicious server could try to:

1. Server replaces Bob's key (KB) with KA
2. Server updates the Merkle tree
3. Server computes new root hash
4. Sends Alice the fake key + valid proof

➤ Technically this is **possible** if the server controls the entire system. So why does Key Transparency still work?

- The protection comes from **global consistency requirements**.
- The server must show **the same Merkle tree root to everyone**.
- If the server cheats, it must maintain **different trees for different users**.
 - Tree for Alice → contains fake key
 - Tree for others → contains real key
 - This is called a **forking attack**.

different versions
of the transparency key.

WHY FORKING IS DETECTABLE

1. Clients compare tree roots.
2. Independent auditors External monitors verify log consistency.
3. **Consistency proofs:** Every new tree must be an extension of the previous tree. So the server **cannot rewrite history.**
4. **Append-Only Property:** A Merkle transparency log must follow:

$$\text{Tree}_{t+1} = \text{Tree}_t + \text{new entries.} \checkmark$$

- It cannot change previous entries. If Bob's key already exists: $\text{Leaf} = \text{Hash}(\text{Bob} \parallel \text{KB})$.
- Replacing it with: $\text{Leaf} = \text{Hash}(\text{Bob} \parallel \text{KA})$, would break the log history.
- Clients verifying consistency proofs would detect it.

← Verify security code
You, 2nd party
name



Symmetric Key.
Who is the
key is authentic.

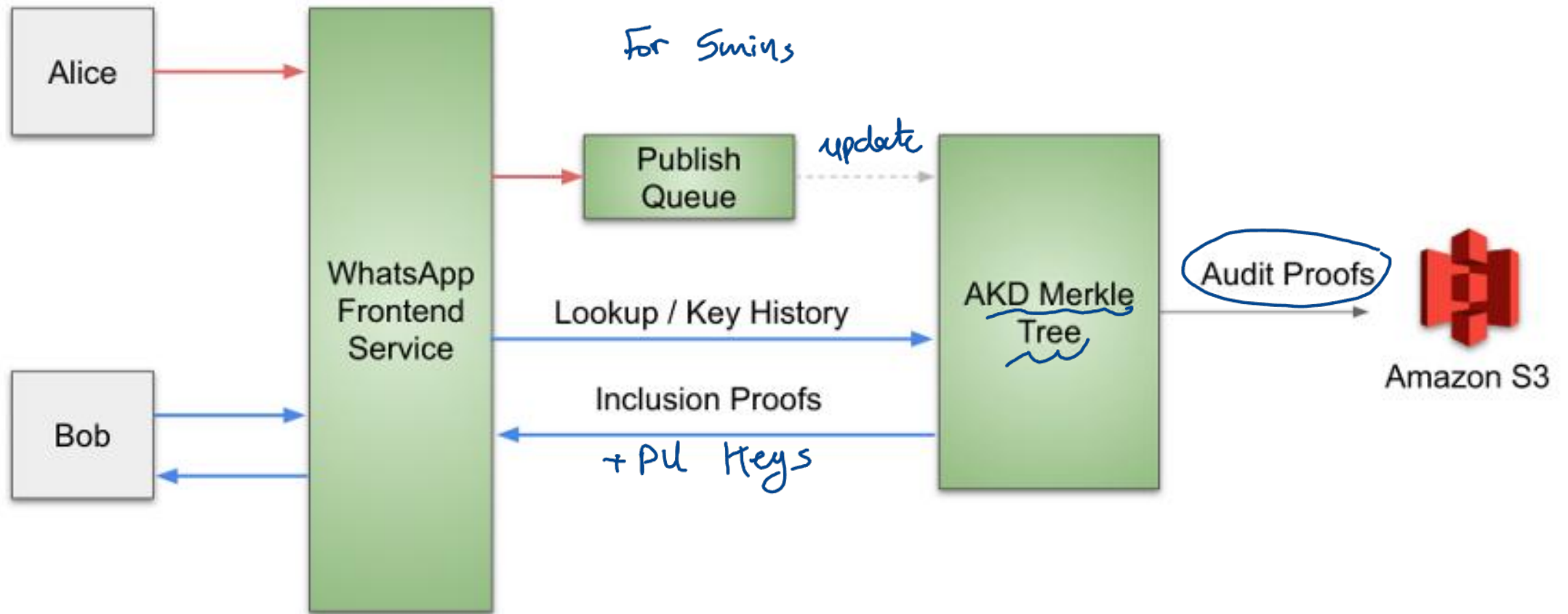
25674 18883 54692 55924
36750 95732 65409 55787
03513 92096 13701 13172

{ To verify that messages and calls with Khalid are end-to-end encrypted, scan or
upload this code on their device. You can also compare the number above
instead. [Learn more](#) }

Scan code

Upload code

ARCHITECTURE



UPDATE MERKLE TREE (KEY CHANGE)

➤ A user's key may change when: They reinstall the app, Change device, Reset their account

➤ Update process:

- The new public key is submitted.
- Same **tree index** in key database (because same user ID).
- but the **leaf value changes.** *Just entry*
- A **new leaf entry** is appended to the log: (userID → new_public_key)
- The Merkle tree is recomputed.
- A new **signed root hash** is published.

AUDITING

- Proofs published publicly
- Anyone can verify. Examples could include: ~~security research institutions~~, ~~universities~~, trusted internet organizations, ~~NGOs~~ or ~~watchdog groups~~ auditors }
- Their tasks: Download the **signed tree roots periodically**. Verify **consistency proofs** between tree versions. Ensure the log is **append-only**.
- Every client running WhatsApp can act as a **lightweight auditor**.
- The system is designed so that **anyone can become an auditor**, because the cryptographic proofs are **publicly verifiable**.

→ anyone can become auditor cuz its publicly available.

MULTIPLE DEVICES

all devices have its own public + private keys & all keys are stored in two DBs.

primary devices

→ Scan QR code becomes multiple devices.

➤ Alice's client fetches all Bob device keys.

▪ Bob devices: Phone → KB1 , Laptop → KB2 , Tablet → KB3

① → servers gives to alice all the public keys for the devices

② ➤ Alice encrypts the message separately for each device. Each device can decrypt its own copy.

3 times encryption.

③ send msg 3 times (overhead)

➤ Why This Design Is Necessary

- Devices to work independently
- Messages to be received even if the phone is offline.

but why this is necessary?

➤ The transparency system records updates when: a new device is added, a device is removed, or a key changes.

- **When the laptop is linked:**

 - Phone authenticates the laptop. [physical authentication]
 - Phone creates a secure encrypted channel with the laptop.
 - Phone re-encrypts message history for the laptop.
 - So the laptop receives a copy of the history encrypted with its own key.
 - The server cannot read the messages because they remain end-to-end encrypted.

Handwritten notes in Urdu:

 - when authenticates
 - what does in this channel?
 - یہ سرور میں کیوں نہیں دیکھتا؟
 - key history کی تاریخ
 - اپنے ہاتھ میں لکھتا ہے۔
 - server is not involved
 - server کو شامل نہیں کیا۔
 - QR code
 - physical authentication
 - public key generated
 - server کو بھیج دیا
 - secure channel
 - key history
 - mobile and laptop

AES KEY REUSE

➤ Symmetric AES encryption is used for the message,

- AES key is exchanged using Diffie Hellman after they fetched the public key

- Each message is encrypted using different key. $K_i = \text{hash}(K_{i-1})$

share some msgs between Alice & Bob
shared key priv. key

x 98 I have multiple devices, &
each one has its own public key

Adv:-

if I knew any key, I won't
know previous msgs, but I
can know next
msgs.

➤ Why they don't reuse one AES key for all devices

- Compromising one device exposes all devices

➤ Even though the message is encrypted multiple times:

- AES encryption is extremely fast,
- messages are small,
- phones handle it easily,
- so the overhead is negligible.

① generate symmetric
keys [AES 256 bits]
using elliptic curve Diffie-
Hellman.

Alice

Bob

pu1



we extract
aes key,
using
public
key.

pu2



pu3



$E(\text{msg})$ [Encryption using symmetric
keys]

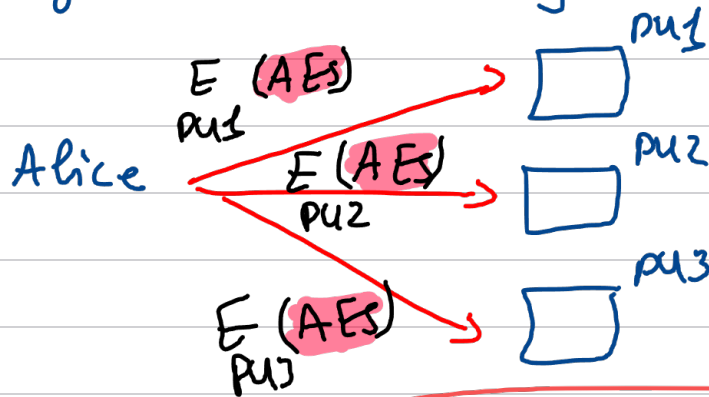
$E_{\text{AES1}}(\text{m}) \rightarrow C1$
 $\text{AES2} \rightarrow C2$
 $\text{AES3} \rightarrow C3$

Next msg
abt the
same
key

$E_{\text{hash}(\text{AES1})}$
 $\text{hash}(\text{AES2})$
 $\text{hash}(\text{AES3})$

Next msg
 $\rightarrow$
 $\text{hash}(\text{hash}(\text{AES}))$
[overhead]

why not generate one aes key?



using AES.

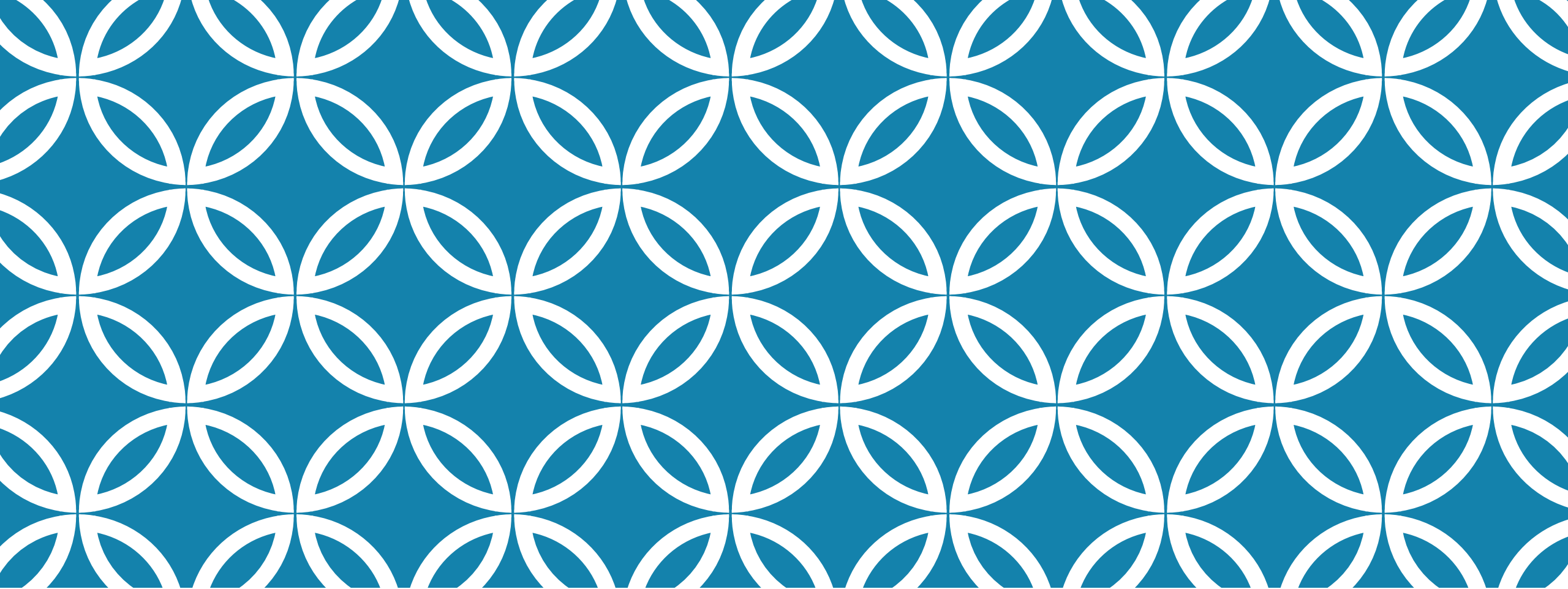
→ one AES adv: encrypt msg one time & send it to the three devices, but send encrypted key 3 times.

disadv:

لو في device ١١ - ١٠ و ١٢
 لو في ١١ Key ١٢ البقية

→ Msgs stored on clear, each msg has its own key can't store them.

hash(hash(...))



END

Questions